🏏 BattingEdge_FYP Project Report

1. Project Overview

- **Goal:** Build a cricket shot classification system (Drive, Pull, Cut, Sweep) using video input.
- **Pipeline (original design):**
    1. **Pose Extraction:** MediaPipe Pose used to extract 33 body landmarks per frame.
    2. **Feature Engineering:** 99 features (x, y, z for each landmark).
    3. **Model Training:** LSTM/Bi-LSTM trained on sequences of 50 frames × 99 features.
    4. **Inference:** inference.py loads trained model, processes new video clips, and predicts shot class.
- **Artifacts:**
    - Notebooks: 28 (utilities, debugging, feature engineering, training, inference demos).
    - Models: V7 and V8 versions (shot_model_V8_best.keras, shot_brain_V8.keras, scalers, encoders).
    - Dataset: dataset_v7_clean with ~1,500 videos.
    - Features: dataset_v7_99feat with ~1,600 .npy files.

---

2. Issues Encountered (Inference Phase)

a. Class Imbalance

- Training set: Drive (381) vs Sweep (140).
- Result: Model biased heavily toward "Drive."
- Inference outputs collapsed to Drive predictions even for other shots.

b. Short/Noisy Clips

- Inference pipeline expected 50 frames per clip.
- Many test/demo clips had fewer frames (e.g., 29 or 31).
- LSTM input mismatch → poor predictions or errors.

c. Feature Limitations

- Only 99 raw skeleton coordinates used.
- Missing biomechanical context (bat angle, stance width, wrist velocity).
- Evaluators could record real-time videos without bats → model struggled.

d. Dataset Organization

- Multiple notebook versions, cluttered folders.
- Validation split missing/misnamed.
- Demo/test videos scattered outside dataset folders.

---

3. Corrective Strategy (Audit 2.0 → V8p)

Step 1 — Dataset Augmentation & Balancing

- Created dataset_v8_balanced_videos.
- Augmented underrepresented classes (Sweep, Cut) with flips, brightness, speed changes, temporal jitter.
- Balanced train set: ~380 videos per class.
- Validation split fixed (val folder).
- Test set left untouched (real distribution).

Step 2 — Feature Engineering V8p

- Regenerate features with:
  - 99 skeleton coords.
  -
    - Wrist velocity (motion intensity).
  -
    - Knee angle (lower body stability).
  -
    - Bat angle proxy (arm vs shoulder line).
  -

        - Stance width (feet distance).

- Resample all clips to 50 frames × 103 features.
- Save to data/features/dataset_v8p.
- Fit and save new scaler (shot_scaler_V8p.pkl).

Step 3 — Model Training V8p

- Train Bi-LSTM with:
    - Balanced dataset.
    - Class weights to handle residual imbalance.
    - Dropout for regularization.
- Save artifacts:
    - shot_model_V8p_best.keras
    - shot_brain_V8p.keras
    - shot_encoder_V8p.pkl
- Evaluate with per-class precision/recall/F1 and confusion matrix.

Step 4 — Inference Update

- Update inference.py to:
    - Load new V8p artifacts.
    - Handle variable-length clips (resample to 50 frames).
    - Apply biomechanical features.
- Test on demo videos → expect more balanced predictions.

---

4. Key Information & Strategies

- **Audit 2.0 Findings:**
    - 28 notebooks, 12 models, 1,521 videos, 1,606 features.
    - Environment stable (TensorFlow, Keras, Mediapipe, OpenCV).
    - Issues: dataset imbalance, cluttered notebooks, missing validation.

- **Strategies Applied:**
  - **Balancing:** Augment minority classes to equalize training distribution.
  - **Feature Enrichment:** Add biomechanical features for robustness.
  - **Resampling:** Normalize all clips to fixed 50-frame sequences.
  - **Validation Fix:** Ensure proper val split for monitoring.
  - **Audit-driven workflow:** Each step verified with counts and summaries.
- **Expected Outcomes:**
  - Improved per-class accuracy (esp. Sweep & Cut).
  - More reliable inference on short/noisy clips.
  - Cleaner project structure for reproducibility.
  - Scaler + encoder ensure consistent preprocessing.

---

5. Next Steps

1. Finish **Step 2 (Feature Engineering V8p)** — currently running.
2. Proceed to **Step 3 (Training V8p)** — train Bi-LSTM with new features.
3. Update and test **inference.py** with V8p artifacts.
4. Archive old V7/V8 models and notebooks, keep only core pipeline files.
5. Document official workflow: Pose → Features → Training → Inference.

📌 Summary

The BattingEdge_FYP project started as a cricket shot classifier using MediaPipe + LSTM. The main issue was **biased inference (always predicting Drive)** due to dataset imbalance and limited features. Through **Audit 2.0**, we identified gaps and launched a corrective pipeline (V8p): balanced dataset, enriched features, retraining, and inference update. This structured approach ensures reproducibility, robustness, and fair evaluation across all shot classes.
